Timur Shakirov

Fullstack developer. Language models in product systems.

20 · Saint Petersburg · remote +7 981 691-94-36 edtimyr@gmail.com @TimaxLacs

github.com/TimaxLacs github.com/TimaxHack github.com/timaxlax npmjs.com/~timax

The main account is `TimaxLacs`. Part of the production code was written under `TimaxHack`: most of the commits to the `deep-assistant` LLM gateway sit there. A single account shows about a third of the work, so all of them are listed.

SUMMARY

An engineer who embeds language models into working products. I designed and wrote the first version of an LLM gateway that became the infrastructure of a commercial AI aggregator: routing between providers, fallback, token-based billing. For the past year I have been running an e-commerce platform under contract: storefront, catalogue, multilingual product cards, generation queues, deployment.

I am interested in orchestrating and embedding neural networks - language models and computer vision - into projects and everyday tasks, including automation.

SKILLS

LANGUAGES	Python, TypeScript, JavaScript
FRONTEND	React, Vite, Tailwind, Capacitor (iOS and Android builds)
BACKEND	Node.js, Express, FastAPI, GraphQL (Hasura), REST, PostgreSQL / PostGIS, Redis, SQLite, MinIO
AI	routing between providers and fallback, token billing, RAG and MCP, text and image generation queues, reasoning pipelines, speech recognition and synthesis
PRODUCTION	Docker, nginx, systemd, Linux, Git, CI, DNS and mail configuration
ENGLISH	technical, minimal level

WORKING WITH MODELS

Since 2024 language models have been part of daily development. Environments: Cursor, Claude Code, ClawCode. Autocomplete is on at all times.

I fix research, constraints and acceptance criteria myself first. Then I fill in the solution myself or with a model. At the planning stage I look up tools and libraries I do not yet know, when they might help.

Before working with a model I write a short set of rules into the repository: how to run checks, what not to touch. The same constraints go into CI and hooks so they cannot be skipped. Code written with a model follows a narrow cycle: plan, edit, run tests and linters. The cycle is run by agents from cursor-agent-workflows : analysis, plan, implementation and verification are split; verification runs separately and does not accept the code the implementer just wrote. File scope is sealed in advance; parallel edits are isolated.

Tests the model wrote together with the implementation are not enough: I check negative and edge cases, secrets and dependencies. I always review the resulting code myself and do not hand it off unreviewed.

EXPERIENCE

E-commerce platform (client project, under NDA)

December 2025 - present

Contract work: frontend, geo module, mobile builds, adjacent backend and deployment

A marketplace with a catalogue and maps. React, TypeScript, Hasura GraphQL, eight services, PostgreSQL / PostGIS, Redis, MinIO, Docker, Capacitor. I run the client application and take part in design decisions; on the backend I own payments, balance and event handlers. Product details are under NDA, the modules I can discuss freely.

- **Storefront:** catalogue, product page, search with autocomplete and filters, registration of several account types with MFA, a UI component library.
- **RU/EN bilingual support and automatic translation of product cards:** dictionaries moved out of the components, translation runs through a hook, a server route and a CLI script that processes the catalogue in batches.
- **Catalogue ingestion pipeline:** sources behind a common contract, deduplication by external id, barcode and attribute fingerprint. Models run after collection in separate queues - text and images. Card facts are never rewritten by a model, otherwise deduplication breaks; a failed worker leaves the product on the storefront with its source data.
- **Wallet and orders:** balance and payment services, external acquiring, status change handlers. Capacitor builds for iOS and Android, complaints and blocking for UGC. Deployment via Docker and systemd, database migration to PostGIS.

> #### Geo module
>
> A single interface over Yandex Maps, Google Maps and 2GIS. Map, marker and zone components know nothing about the provider: engines accept a command and pass it to an adapter, the adapter translates unified data into calls of its own SDK. Switching provider is one configuration parameter.
>
> - Zone geometry: buffer zones along a route with smooth transitions into circular marker zones through quadratic Bézier curves, polygon union with holes and multipolygons, geometry validation, caching of computations.
> - Draggable markers and user-defined zones; changes made on the map propagate into application state through callbacks. Geocoding moved to a server route so that API keys never reach the browser.
> - Geometry stored in PostGIS, radius search, multilingual labels, smoke tests and documentation for extracting the module into a standalone application.

ManyAir - international money transfer platform

August 2025 - May 2026

Client project: frontend and adjacent backend work

A microservice system around Hasura GraphQL: router, event handler, mailing service, Telegram bot, document generator, currency rates; PostgreSQL, Redis, MinIO.

- **Resilient notification delivery:** when a channel is unavailable the status returns “not sent” and the invitation is not marked as delivered.
- **Data isolation between roles:** query cache reset on user change, restricted subscriptions to deal details.
- Removal of the legacy invitations domain from the system: tables, Hasura permissions, routes, handlers, generated types.
- Server-side maintenance: service deployment, failure analysis, mail and DNS configuration for the project.

Deep.Assistant - AI model aggregator

2024-2025

Founder, backend and DevOps · team of three · @DeepGPTBot, channel @gptDeep - 1,553 subscribers

A commercial product: chat with models, generation of images, music and voice, an API gateway, mini-apps in Telegram and VK.

LLM gateway - github.com/deep-assistant/api-gateway

I designed and wrote the first version; it grew out of my personal repository resale-chatgpt-azure into the infrastructure that carries every request in the product. Main author of the code: HTTP server, model configuration, token manager, database layer.

- About 37 models from six providers behind a single OpenAI-compatible interface.
- **Degradation table instead of a plain retry:** each model has a chain of 12-16 substitutes, selected so that the answer remains usable for the same scenario.
- **A single account across incomparable price lists:** every model has its own coefficient for converting tokens into the internal currency, from 1.7 to 40. The product calculates the cost of a request and the markup.
- **Idempotent transfers between accounts:** exclusive locks with stale-lock detection, atomic writes with read-back verification, recovery of unfinished operations at startup.
- **Speed of adding new models:** o1-preview and o1-mini were connected on 27 September 2024, deepseek-reasoner on 30 January 2025, each time together with pricing and fallback chains.

I maintain the repository and review changes from the second developer.

Telegram bot - github.com/deep-assistant/telegram-bot

A multi-model assistant built on aiogram. My parts:

- **Transfers of the internal currency between users.** A state-machine dialogue: recipient, amount, confirmation, cancellation, operation history. I wrote the module from scratch.
- User synchronisation service: lazy profile loading on first request and bulk reconciliation with limited concurrency, so as not to hit Telegram rate limits.
- Audio transcription mode with a separate pricing coefficient, custom system messages.
- Behaviour in group chats: replies only when addressed, parsing of mentions in photo and document captions, export of the dialogue history as a file.
- Payment via Telegram Stars, separation of the TEST and PROD environments, asynchronous provider availability checks.

Matreshka - public website of an AI/e-commerce platform

2026

Client project: frontend and site launch · matreshka.club

The public website of the platform: React, Vite, TypeScript, Tailwind, brand identity. Deployed on nginx with SSL, configured DNS and mail on the domain. Second project for the same client.

THE DEEP.FOUNDATION ECOSYSTEM

Associative data model: contributor since 2022

2022 - present

github.com/deep-foundation · member of the organisation

Deep.Foundation is building a store in which the only entity is a link: a tuple of an identifier, a type and references to other links. A link can reference a link, so a single table describes the data, the schema and the handlers alike.

- **`deep-files-sync` - a projection of the link graph onto the file system.** My package, 13 versions. Links are laid out into directories, the value sits in a text file next to the object, and folder nesting expresses the relations: a path through directories is isomorphic to a database query. Edits on both sides are kept in sync, a subscription to changes is materialised as a file, and deleting the file means unsubscribing. The point is that the contents of the database can be read and edited with ordinary tools - `grep`, an editor, git. github.com/TimaxLacs/deep-files-sync
- **AI tooling packages in the official scope.** I released versions of `@deep-foundation/ai`, `ai-models`, `ai-templates`, `ai-tasks`, `4o-bot`, `chatgpt-azure-deep`, `function_generation`, `tests_generation`. Such a package is not a library but a dump of a configured link graph that is deployed inside someone else's instance.
- **Bot dialogues as a graph.** I implemented writing conversations into the associative store: dialogue, message, author and reply are expressed as link types, with no separate table per entity. The solution ran in production for about six weeks and was then replaced by file storage - the complexity did not pay off.
- Co-author of the Russian-language theoretical documentation for the model, about 1,000 lines. Filed around 36 issues - mostly bug reports on platform deployment, with tracebacks.

Meta-theory of links - co-author

2026

A formal system whose only primitive is a directed link and in which data is addressed by associative numbers. I take part in developing the theory and in the discussions; the document repository is maintained by the author of the theory.

atlas.metasteme.dev · github.com/netkeep80/anum_docs

The theory has a profile in the Atlas of Limit Theories of the Runet.

AI CASES

Pipelines and agent workflows

2025-2026

- **test-resorning** - a reasoning pipeline on models without a built-in reasoning mode: intent prediction, step-by-step solution, adversarial critique of its own answer, refinement, synthesis. A LangChain variant with Tree of Thoughts and self-consistency. April 2025; the behaviour is reproduced by prompt orchestration rather than by calling a reasoning model.
- **lightrag-mcp** - an MCP server for a self-hosted LightRAG deployment: graph and vectors, search modes, reranking, answers with links to sources. The configuration is validated with zod, tests run on the built-in `node --test`. Connected to my working editor as persistent memory across projects.
- **ai-post-generator** - a content pipeline with separate AI moderation: draft, review by a second call, up to three rounds of edits, publication. State is kept in SQLite so it survives a process crash; publishers for Telegram, VK and MAX are extracted into strategies.
- **cursor-agent-workflows** - a set of agent workflows in 1,900 lines of specifications: a safe code change pipeline of five agents with sealed task boundaries and parallel streams, project bootstrap, and a research workflow. I use it daily.

OPEN SOURCE AND PACKAGES

Voice: local stack and npm packages

2024-2025

github.com/TimaxLacs/InnerEcho · npmjs.com/~timax

- Local voice assistant InnerEcho: speech recognition, a language model, synthesis with voice cloning, automatic setup of audio devices. Comparison of synthesis models on a Russian corpus. A research prototype.
- `xtts-v2-js` - a wrapper over XTTS-v2 with voice cloning in 17 languages: it provisions the Python environment itself, so the package installs like an ordinary Node.js dependency. 7 versions.

- `zonosjs` and `zonos-client` - a client for the local Zonos speech synthesis server, 10 versions in total.
- `@timax/passport-authorization` and related packages - registration and confirmation by email.

tg-runner - bot orchestration

January 2026

A set of four components

A state-machine orchestrator for launching bots on Redis, a worker that starts bots in Docker, a published SDK for custom workers, and a CLI. Docker Compose, pytest, documentation.

github.com/TimaxLacs/ - tg-runner-orchestrator · tg-runner-worker · tg-runner-worker-sdk · tg-runner-cli

PROJECTS

Bots for the MAX messenger

2026

github.com/TimaxLacs/max-bot

Multimodal photo processing by a model, a per-user quota, Docker, integration with an asynchronous AI worker and a fallback scenario when the backend is unavailable.

vk-tg-parser

2026

github.com/TimaxLacs/vk-tg-parser

A pip package: a single parser for Telegram over MTProto and VK over the API, with a shared data model and real-time monitoring.

Hashtag auto-commenter

2026

github.com/TimaxLacs/tg-hashtag-commenter

Commenting by hashtags in Telegram and VK with rate limiting and deduplication. Live run in May 2026.

ai-lector

2025

github.com/TimaxLacs/ai-lector

A bot that reworks audio lectures into the required format. I built a local Telegram Bot API server with cmake to get around the file size limit.

anti-cpam-ai

2025

github.com/TimaxHack/anti-cpam-ai

An anti-spam bot: a model called through OpenRouter assigns each message a spam score from 1 to 10.

Orbiteer

2025

github.com/TimaxLacs/Orbiteer

A risk analyser for space missions: collision probability calculation, 3D visualisation of orbits with three.js, an API on Sanic.

EARLY WORK

GEFFEST - manipulator glove

2021-2022

github.com/TimaxHack/GEFFEST

A glove that reproduces hand movements on a manipulator. The position of the fingers and the wrist is read from sensors and computed with quaternions - this removes the breakdowns that Euler angles cause when the wrist flips over. The glove and the manipulator communicate over radio with a custom compact protocol, so latency does not grow with the number of channels.

On the manipulator side, servo movement is smoothed, otherwise hand tremor turns into jerks in the mechanics. I soldered the circuit and the prototype boards myself. Separately, I wrote an Android voice control application in Java with Vosk - recognition works without an internet connection.

Early bots and computer vision

2021

Library bot - a map of libraries holding a given book, a team project. A Lake Onega freezing bot - determining the freezing level from street camera frames with computer vision.

Ventures that did not take off

2023-2025

ArtApollo - a studio where orders were drawn by neural networks. I built a team of up to 30 people, took the first paid orders and delivered a corporate order for 12 people. The project is closed and the team has disbanded.

Cyber Vestis - a prototype of smart clothing with climate control: software for a touch display, body temperature telemetry. We reached a working prototype but not investment.

EDUCATION

Secondary school, 11 years (Russian general education)

No degree in the field. I have been programming since 2021 and running commercial projects since 2024; everything listed above was built in practice and is available in public repositories.

timaxlacs.github.io edtimyr@gmail.com @TimaxLacs +7 981 691-94-36